

QUESTION : String Problem

Description

In a certain encrypted message which has information about the location(area, city), the characters are jumbled such that first character of the first word is followed by the first character of the second word, then it is followed by second character of the first word and so on

In other words, let's say the location is bandra,mumbai

The encrypted message says 'bmaunmdbraai'.

Sample Input:

bmaunmdbraai

Sample Output:

bandra,mumbai

Let's say the size or length of the two words wouldn't match then the smaller word is appended with # and then encrypted in the above format.

With this in mind write a code to identify the right location and print it as place,city.

```
1 input_str = input()
2 print(input_str[::2].strip('#') + ',' + input_str[1::2].strip('#'))
```

QUESTION : Digit or Alphabet

Description

Write a program to display whether the input is a digit or a letter of the alphabet.

Input:

A digit or a letter of the alphabet

Output:

Displays whether the given output is an integer or a letter of the alphabet

Sample input:

1

Sample output:
Integer

Sample input:
b

Sample output:
Alphabet

```
1 if input().isdigit(): print("Integer")
2 else: print("Alphabet")
```

QUESTION : ASCII values

Description

Write a program to accept a character and display its next and previous character.

Hint: Make use of Ascii values here.

Input:
A character

Output:
Previous character and the next character of the given character

Sample input:
D

Sample output:
C
E

Sample input:
8

Sample output:
7
9

Sample input:

@

Sample Output:

?

A

```
1 char = input()
2 print(chr(ord(char)-1))
3 print(chr(ord(char)+1))
```

QUESTION : No Vowels

Description

Write a program to accept a string from the user, delete all vowels from the string and display the result.

Input:

A string

Output:

A string with vowels removed

Sample input:

Upgrad

Sample output:

pgrd

Sample input:

Python Programming

Sample output:

Pythn Prgrmmng

```
1 string = input()
2 for i in string:
3     if i in 'aeiouAEIOU': string = string.replace(i, "")
4 print(string)
```

QUESTION : Valid triangle

Description

Write a program to accept three sides of a triangle as the input and print whether the triangle is valid or not.

(A triangle is valid if the sum of any two sides is greater than the third side.)

Input:

Three sides of a triangle separated by a space

Output:

Whether the given triangle is "Valid" or "Invalid"

Sample input:

3 4 5

Sample output:

Valid

Sample input:

1 4 5

Sample output:

Invalid

```
1 a,b,c= input().split()
2 if int(a) + int(b) > int(c) and int(b) + int(c) > int(a) and int(c) + int(a) > int(b): print("Valid")
3 else: print("Invalid")
```

QUESTION : Multiplication Table

Description

Write a program to display the multiplication table of a given number.

Input:

A positive integer

Output:

Multiplication table

Sample input:

5

Sample output:

5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50

```
1 n=int(input())  
2 for i in range(1, 11): print(n, '*', i, '=', n*i)
```

QUESTION : Character frequency

Description

Write a program to accept a string value from the user and accept a char value from the user and find out the total occurrence of the char value in the string value. Note that the count is not case-sensitive

Input:

A string and a character whose occurrence is to be found

Output:

An integer

Sample input:

Python Programming

P

Sample output:

2

Sample input:

This is my first program

t

Sample output:

2

```
1 input_string=input().lower()
2 input_char=input().lower()
3 print(input_string.count(input_char))
```

QUESTION : Sum of digits

Description

Write a program to calculate the sum of the digits of a given number

Input:

An n digit number

Output:

Sum of the digits

Sample input:

983

Sample output:

20

Sample input:

5241

Sample output:

12

```
1 n=input()  
2 sum = 0  
3 for i in n: sum += int(i)  
4 print(sum)
```

QUESTION : Count the digits

Description

Write a program to accept a number from the user and count the zeros, odd digits and non-zero even digits from the entered number.

Input:

A positive integer of n digits

Output:

Three integers representing the occurrences of zeros, odd digits and non-zero even digits from the entered number.

Sample input:

1030

Sample output:

Number of odd digits: 2

Number of non-zero even digits: 0

Number of zeros: 2

```
1 n = int(input())
2 odd_count = 0
3 non_zero_even_count = 0
4 for i in str(n):
5     if i in '13579': odd_count += 1
6     elif i in '2468': non_zero_even_count += 1
7 print('Number of odd digits:', odd_count)
8 print('Number of non-zero even digits:', non_zero_even_count)
9 print('Number of zeros:', str(n).count('0'))
```

QUESTION : String manipulation

Description

Write a program that takes a string as the input and does the following:

- Removes the numbers, special characters
- Converts uppercase letters to lowercase letters, and vice versa

Input:

A string

Output:

A string with numbers, special characters removed, upper and lower cases swapped

Sample input:

Programming1234

Sample output:

pROGRAMMING

Sample input:

Programming is 100% fun

Sample output:

pROGRAMMINGISFUN

```

1 input_string=input()
2 for i in input_string:
3     if i.isalpha() == False: input_string = input_string.replace(i, "")
4     elif i.islower(): input_string = input_string.replace(i, i.upper())
5     else: input_string = input_string.replace(i, i.lower())
6 print(input_string)

```

QUESTION : List_remove_append

Description

Remove SPSS from input_list=['SAS', 'R', 'PYTHON', 'SPSS'] and add 'SPARK' in its place.

Sample Input:

['SAS', 'R', 'PYTHON', 'SPSS']

Sample Output:

['SAS', 'R', 'PYTHON', 'SPARK']

```

1 import ast,sys
2 input_list = (sys.stdin.read()).split(',')
3 input_list.remove('SPSS')
4 input_list.append('SPARK')
5 print(input_list)

```

QUESTION : List to String

Description

Convert a list ['Pythons syntax is easy to learn', 'Pythons syntax is very clear'] to a string using '&'. The sample output of this string will be:

Pythons syntax **is** easy to learn & Pythons syntax **is** very clear

Note that there is a space on both sides of '&' (as usual in English sentences).

```
1 import ast,sys
2 input_str = (sys.stdin.read()).split(',')
3 string_1 = input_str[0] + ' & ' + input_str[1]
4 print(string_1)
```

QUESTION : String Split

Description

Split the string `input_str = 'Kumar_Ravi_003'` to the person's second name, first name and unique customer code. In this example, `second_name= 'Kumar'`, `first_name= 'Ravi'`, `customer_code = '003'`.

A sample output of the input 'Kumar_Ravi_003' is:

```
Ravi
Kumar
003
```

Note that you need to **print in the order** first name, last name and customer code.

```
1 import ast,sys
2 input_str = sys.stdin.read()
3 list1 = input_str.split('_')
4 first_name = list1[1]
5 second_name = list1[0]
6 customer_code = list1[2]
7 print(first_name)
8 print(second_name)
9 print(customer_code)
```

QUESTION : Tuple

Description

Add the element 'Python' to a tuple `input_tuple = ('Monty Python', 'British', 1969)`. Since tuples are immutable, one way to do this is to convert the tuple to a list, add the element, and convert it back to a tuple.

Sample Input:

('Monty Python', 'British', 1969)

Sample Output:

('Monty Python', 'British', 1969, 'Python')

```
1 import ast,sys
2 input_str = sys.stdin.read()
3 input_tuple = ast.literal_eval(input_str)
4 tuple_2 = input_tuple + ('Python',)
5 print(tuple_2)
```

QUESTION : Sets Practice**Description**

Let's say you have two lists A and B. Identify the elements which are common in the two lists A and B and return them in a sorted manner. For example

Sample Input :

A = [5, 1, 3, 4, 4, 5, 6, 7]

B = [3, 3, 5, 5, 1, 7, 2]

Sample Output:

[1,3,5,7]

If you observe the sample output here you can see that

1. Though value 5 is repeated twice in both lists, in the final output it is present only once.
2. The values are returned in a sorted manner with values increasing.

```
1 import ast
2 input_list = ast.literal_eval(input())
3 print(sorted(list(set(input_list[0]) & set(input_list[1]))))
```

QUESTION : Dictionary Coding

Description

Write code to fetch the profession of the employee with Employee id - 104 from an employee input given in the form of a dictionary where key represent employ id and values represent the name, age, and profession (in the same order).

Sample input:

```
Employee_data = { 101:['Shiva', 24, 'Content Strategist'], 102:['Udit',25,'Content Strategist'], 103:['Sonam', 28,'Sr Manager'], 104:['Ansari',29,'Product Lead'], 105:['Huzefa',32,'Project Manager' ]}
```

Sample output:

'Product Lead'

Note: Assume that employee data would be available in the data provided to you

```
1 import ast
2 print(ast.literal_eval(input())[104][2])
```

QUESTION : Dict_Error

Description

From a Dictionary input_dict={'Name': 'Monty', 'Profession': 'Singer' }, get the value of a key 'Label' which is not a part of the dictionary, in such a way that Python doesn't hit an error. If the key does not exist in the dictionary, Python should return 'NA'.

Sample Input:

```
{'Name': 'Monty', 'Profession': 'Singer' }
```

Sample Output:

NA

```
1 import ast
2 print(ast.literal_eval(input()).get('Label', 'NA'))
```

QUESTION : List of Values in a Dictionary.

Description

Create a SORTED list of all values from the dictionary input_dict = {'Jack Dorsey' : 'Twitter' , 'Tim Cook' : 'Apple','Jeff Bezos' : 'Amazon' , 'Mukesh Ambani' : 'RJIO'}

Sample Input:

{'Jack Dorsey' : 'Twitter' , 'Tim Cook' : 'Apple','Jeff Bezos' : 'Amazon' , 'Mukesh Ambani' : 'RJIO'}

Sample Output:

['Amazon', 'Apple', 'RJIO', 'Twitter']

```
1 import ast
2 print(sorted(ast.literal_eval(input()).values()))
```

QUESTION : List sum

Description

Suppose you want to know the total score of the Indian cricket team in a given match. To do so, your task is to find the sum of all the scores of the Indian team players. The scores are provided as a list, with each element as an individual score of the players. Also, there is a condition that if the number of elements in the list is more than 11, then it is an invalid input and the output should be -1.

Input - List

Output - An integer

Sample Input : [11, 13, 101, 14, 33, 141]

Sample Output : 313

Sample Input : [11, 13, 101, 14, 33, 141, 12, 144, 54, 67, 8, 11]

Sample Output : -1

Sample Input : [100, 80, 30, 10, 0, 1, 2, 0]

Sample Output : 223

```
1 import ast
2 input_list = ast.literal_eval(input())
3 if len(input_list) <= 11: print(sum(input_list))
4 else: print(-1)
```

QUESTION : Length of list elements

Description

Given a list of strings, write a program to find the number of strings whose length is greater than or equal to K, where K is a positive integer.

Input - List of strings and an integer

Output - Integer

Sample Input :

[Mumbai, Hyderabad, Calicut, Chennai]

9

Sample Output: 1

Sample Input :

[Datascience, Data Analyst, Programmer, Manager]

8

Sample Output : 3

```
1 import ast
2 input_list = ast.literal_eval(input())
3 K = int(input())
4 count = 0
5 for i in input_list:
6     if len(i) >= K: count += 1
7 print(count)
```

QUESTION : Increment list elements

Description

Given a list of strings, increment the value of the numeric strings by 'k'.

Hint: The function [isdigit\(\)](#) may be useful here.

Input - A list in the first line and an integer in the second line

Output - A list

Sample Input :

['Python', '123', 'Data']

4

Sample Output : ['Python', '127', 'Data']

Sample Input :

['upGrad', '1991', 'Mumbai']

0

Sample Output : ['upGrad', '1991', 'Mumbai']

Sample Input :

['Data Science', '100', '10']

10

Sample Output : ['Data Science', '110', '20']

```
1 import ast
2 input_list = ast.literal_eval(input())
3 K = int(input())
4 print([str(int(input_list[i]) + K) if input_list[i].isdigit() else input_list[i] for i in range(len(input_list))])
```

QUESTION : Slicing a list

Description

Given a list of strings and an integer K, write a python code to print all the elements from the Kth position till the end of the list.

Note: Assume that K (a positive integer) will always be less than or equal to the length of the list

Input - A list of strings in the first line and an integer in the second line of the input.

Output - A list

Sample Input :

['Mumbai', 'Delhi', 'Australia', 'Nigeria', 'USA', 'London', 'Canada']

2

Sample Output : ['Delhi', 'Australia', 'Nigeria', 'USA', 'London', 'Canada']

Sample Input :

['Chennai', 'Vizag', 'Austria', 'Germany', 'Japan']

3

Sample Output : ['Austria', 'Germany', 'Japan']

```
1 import ast
2 print(ast.literal_eval(input())[int(input())-1:])
```

QUESTION : Substring with maximum uppercase characters

Description

Given a string, write a Python program to find the largest substring of uppercase characters and print the length of that substring. Check the sample inputs and outputs for a better understanding.

Input - String

Output - String

Sample Input - I lovE PRogrAMMING

Sample Output - 6

Explanation - AMMING is the largest substring with all characters in uppercase continuously

Sample Input - MuMbal is in MAHArashTRA

Sample Output - 4

Explanation - MAHA is the largest substring with all characters in uppercase continuously.

Sample Input - India WOn the WOrLD CUP

Sample Output - 3

Explanation - CUP is the largest substring with all characters in uppercase continuously.

```
1 test_str = input()
2 string2 = ""
3 for i in test_str:
4     if i.islower(): test_str = test_str.replace(i, ' ')
5     else: pass
6 list1 = [i for i in test_str.split(' ')]
7 print(len(max(list1, key = len)))
```

QUESTION : Sorting list of tuples

Description

A class of students attempt an exam in two parts: 'Aptitude' and 'Physics'. The marks of all the students are stored as a list of tuples, and each student's marks (in Aptitude and Physics) are stored in each tuple. Your task is to write a Python program to sort the list of tuples in decreasing order of the Physics scores of the students.

Note - Marks of both the subjects are ranged between 1-100, and no two students have scored the same marks in Physics.

Input - List of tuples

Output - List of tuples

Sample Input - [(45,77), (88,87), (67,98), (33,31)]

In (45,77), which is the first element in the list, 45 and 77 are the scores of a student in aptitude and physics respectively.

Sample Output - [(67,98), (88,87), (45,77),(33,31)]

Sample Input - [(45,23), (45,88), (45,98), (45,44)]

Sample Output - [(45,98), (45,88), (45,44),(45,23)]

Sample Input - [(55,77), (34,90), (67,100), (90,0)]

Sample Output - [(67,100), (34,90), (55,77),(90,0)]

```
1 import ast
2 input_list = ast.literal_eval(input())
3 print(sorted(input_list, key = lambda x : x[1], reverse = True))
```

QUESTION : Divisibility check

Description

Given an integer 'n', your task is to write a Python code to find whether 'n' is divisible by all its digits or not. If they divide the number, then the number 'n' is a happy number. Otherwise, it is a sad number.

The number 'n' may be provided with commas. At first, you have to clean the number (by removing the commas involved) and then check whether the number is happy or sad.

Input - String

Output - String

Sample Input - 2,128

Sample Output - Happy Number

Sample Input - 256

Sample Output - Sad Number

Sample Input - 1124

Sample Output - Happy Number

```
1 n = input()
2 happy = 0
3 sad = 0
4 for i in n:
5     if i.isdigit() == False: n = n.replace(i, '')
6 for i in n:
7     if int(n)%int(i) == 0: happy += 1
8     else: sad += 1
9 if happy > 0 and sad == 0: print('Happy Number')
10 else: print('Sad Number')
```

QUESTION : Updating the dictionary keys

Description

You are provided with a dictionary containing the names of different football clubs as **keys** and the name of the main player of the corresponding team as **values**. When the main player of a team retires, one of their teammates steps up to fill in their role as the main player.

You're also provided with a list which contains the names of the football clubs for which the current main players are retiring and the names of the corresponding new main player. Your task is to update the values in the original dictionary with the names of the new main players.

Input - The input consists of a dictionary with different football clubs' names and their main player. In the next line, you will be provided with a list of the new players who have taken up the main roles.

Output - Dictionary

Sample Input :

```
{Barcelona:'Messi' , 'Real Madrid': 'Benzema', 'PSG': 'Neymar' }  
[['Barcelona', 'Griezmann'], ['PSG', 'Ramos']]
```

Sample Output: {Barcelona:'Griezmann', 'Real Madrid': 'Benzema', 'PSG': 'Ramos'}

Sample Input:

```
{Liverpool:'A', 'Real Madrid': 'B', 'Chelsea': 'C' }  
[[Liverpool, 'R']]
```

Sample Output: {Liverpool:'R', 'Real Madrid': 'B', 'Chelsea': 'C' }

Sample Input :

```
{Atletico Madrid : 'D' , 'Juventus': 'B', 'Chelsea': 'C' }  
[['Juventus', 'G'], ['Chelsea', 'H']]
```

Sample Output: {Atletico Madrid : 'D', 'Juventus': 'G', 'Chelsea': 'H' }

```
1 import ast
2 convert_dictionary = ast.literal_eval(input())
3 convert_list = ast.literal_eval(input())
4 convert_dictionary.update({i[0]:i[1] for i in convert_list if i[0] in convert_dictionary.keys()})
5 print(convert_dictionary)
```

QUESTION : List average

Description

Suppose you are working as a Marketing Lead in a company and you want to recruit new employees in your team (which already consists of 10 members) to carry out the company's major project. You select the employees based on five parameters: A, B, C, D and E. The value of each parameter is represented on a scale of 1 to 10. The present average value of these parameters for your team is given to you in a list (in the same order). You recruit a person only if their scores do not reduce the average score of more than two parameters of your team. If they are recruited, print the output as 'Selected'. If not, print the output as 'Rejected'.

Input - Two lists, where the first list is the average scores of the team in the five parameters, and the second list is the scores of the new employee in all the five parameters. The parameter order is the same for both the lists, which is A, B, C, D and E.

Output - String

Sample Input - ['8', '4', '6', '9', '7']
['1', '1.1', '1.2', '1.2', '2.3']

Sample Output - Rejected

Sample Input - ['10', '5', '6', '9', '7']
['10', '9.8', '7.2', '1.66', '4.3']

Sample Output - Selected

Sample Input - ['8', '5.66', '6.5', '10', '10']
['7', '10', '6', '7', '9']

Sample Output - Rejected

```
1 import ast
2 team_parameters_average_scores = ast.literal_eval(input())
3 employee_paramteres_score = ast.literal_eval(input())
4 negative = 0
5 list1 = [float(element2) - float(element1) for element2, element1 in zip(employee_paramteres_score,team_parameters_average_scores)]
6 for i in list1:
7     if i < 0: negative += 1
8 if negative > 2: print('Rejected')
9 else: print('Selected')
```

QUESTION : Merge dictionaries

Description

Write a python code to merge two dictionaries into a single dictionary.

Input: Two dictionaries, one in each line

Output: A Dictionary

Sample Input :

{'a': 10, 'b': 8}

{'d': 6, 'c': 4}

Sample Output : {'c': 4, 'a': 10, 'b': 8, 'd': 6}

Sample Input :

{'a': 110, 'b': 88}

{'d': 62, 'c': 44}

Sample Output : {'a': 110, 'b': 88, 'd': 62, 'c': 44}

```
1 import ast
2 dict1 = ast.literal_eval(input())
3 dict2 = ast.literal_eval(input())
4 dict1.update({key:value for key,value in dict2.items() if key not in dict1.keys()})
5 print(dict1)
```

QUESTION : If-Else

Description

Write a code to check if the string in input_str starts with a vowel or not. Print capital YES or NO.

For example, if input_str = 'analytics' then, your output should print 'YES'.

Sample Input:

alpha

Sample Output:

YES

```
1 import ast,sys
2 input_str = sys.stdin.read()
3 if input_str[0] in 'aeiouAEIOU': print('YES')
4 else: print('NO')
```

QUESTION : Iterations

Description

You are given a list of string elements and asked to return a list which contains each element of the string in title case or in other words first character of the string would be in upper case and remaining all characters in lower case

Sample Input:

['VARMA', 'raj', 'Gupta', 'SaNdeeP']

Sample Output

['Varma', 'Raj', 'Gupta', 'Sandeep']

```
1 import ast
2 print([i.capitalize() for i in ast.literal_eval(input())])
```

QUESTION : List Comprehension

Description

You are given an integer 'n' as the input. Create a list comprehension containing the squares of the integers from 1 till n^2 (including 1 and n), and print the list.

For example, if the input is 4, the output should be a list as follows:

```
[1, 4, 9, 16]
```

The input integer 'n' is stored in the variable 'n'.

```
1 print([i ** 2 for i in range(1, int(input())+1)])
```

QUESTION : List Comprehensions

Description

Extract the words that start with a vowel from a list `input_list=['wood, old, apple, big, item, euphoria]` using list comprehensions.

Sample Input:

['wood','old','apple','big','item','euphoria']

Sample Output:

['old', 'apple', 'item', 'euphoria']

```
1 import ast
2 print([i for i in ast.literal_eval(input()) if i[0] in 'aeiouAEIOU'])
```

QUESTION : Function**Description**

Create a function squared(), which takes x and y as arguments and returns the x^y value. For e.g., if $x = 2$ and $y = 3$, then the output is 8.

Sample Input:

['6','7']

Sample Output:

279936

```
1 import ast
2 input_list = ast.literal_eval(input())
3 def squared(x, y):
4     return x**y
5 print(squared(int(input_list[0]), int(input_list[1])))
```

QUESTION : Lambda**Description**

Create a lambda function 'greater', which takes two arguments x and y and return x if $x > y$ otherwise y.

If $x = 2$ and $y = 3$, then the output should be 3.

Sample Input:

['9','3']

Sample Output:

9

```

1 import ast,sys
2 input_list = ast.literal_eval(input())
3 a = input_list[0]
4 b = input_list[1]
5 greater = lambda a, b : int(a) if a > b else int(b)
6 print(greater(a, b))

```

QUESTION : Map Function

Description

Using the function Map, count the number of words that start with 'S' in input_list.

Sample Input:

['Santa Cruz','Santa fe','Mumbai','Delhi']

Sample Output:

2

```

1 import ast
2 print(list(map(lambda x : x[0] == 'S', ast.literal_eval(input()))).count(True))

```

QUESTION : Map Function

Description

Create a list 'name' consisting of the combination of the first name and the second name from list 1 and 2 respectively.

For e.g. if the input list is:

```
[ ['Ankur', 'Avik', 'Kiran', 'Nitin'], ['Narang', 'Sarkar', 'R', 'Sareen'] ]
```

the output list should be the list:

```
['Ankur Narang', 'Avik Sarkar', 'Kiran R', 'Nitin Sareen']
```

Note: Add a space between first name and last name.

```
1 import ast
2 input_list = ast.literal_eval(input())
3 print(list(map(lambda x, y : x + ' ' + y, input_list[0], input_list[1])))
```

QUESTION : Filter Function

Description

Extract a list of numbers that are multiples of 5 from a list of integers named input_list.

Sample Input:

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50]

Sample Output:

[5, 10, 15, 20, 25, 30, 35, 40, 45, 50]

Note: Use the filter() function.

```
1 import ast
2 print(list(filter(lambda x : x%5 == 0, ast.literal_eval(input()))))
```

QUESTION : Filter Function

Description

You are given a list of strings such as input_list = ['hdjk', 'salsap', 'sherpa'].

Extract a list of names that start with an 's' and end with a 'p' (both 's' and 'p' are lowercase) in input_list.

Sample Input:

['soap','sharp','shy','silent','ship','summer','sheep']

Sample Output:

['soap', 'sharp', 'ship', 'sheep']

Note: Use the filter() function.

```
1 import ast
2 print(list(filter(lambda x : x[0] == 's' and x[-1] == 'p', ast.literal_eval(input()))))
```

QUESTION : Reduce Function**Description**

Using the Reduce function, concatenate a list of words in input_list, and print the output as a string.

If input_list = ['I','Love','Python'], the output should be the string 'I Love Python'.

Sample Input:

['All','you','have','to','fear','is','fear','itself']

Sample Output:

All you have to fear is fear itself

```
1 import ast
2 from functools import reduce
3 print(reduce(lambda x, y : x + ' ' + y, ast.literal_eval(input())))
```

QUESTION : Reduce Function**Description**

You are given a list of numbers such as input_list = [31, 63, 76, 89]. Find and print the largest number in input_list using the reduce() function.

Sample Input:

[65,76,87,23,12,90,99]

Sample Output:

99

```
1 import ast
2 from functools import reduce
3 print(reduce(lambda x, y : x if x > y else y, ast.literal_eval(input())))
```

QUESTION : Even numbers in a list

Description

Given a list of integers, write a python code to find all the even numbers present in the list.

Note: Try using lambda and filter functions to solve the problem.

Input: A list of integers

Output: A list of integers

Sample input: [5, 7, 22, 97, 54, 62, 77, 23, 73, 61]

Sample output: [22, 54, 62]

Sample input: [500, 789, 2112, 9097, 5894, 9062, 8977, 1223, 773, 600]

Sample output: [500, 2112, 5894, 9062, 600]

```
1 import ast
2 print(list(filter(lambda x : x%2 == 0, ast.literal_eval(input()))))
```

QUESTION : Triple the list elements

Description

Given a list of integers, write a python code to triple all the values in the list.

Note: Try using lambda and map functions to solve the problem.

Input: A list of integers
Output: A list of integers

Sample input: [5, 7, 22, 97, 54, 62, 77, 23, 73, 61]
Output: [15, 21, 66, 291, 162, 186, 231, 69, 219, 183]

Sample input: [15, 71, 2, 9, 5, 6, 11, 3, 7, 6]
Output: [45, 213, 6, 27, 15, 18, 33, 9, 21, 18]

```
1 import ast
2 print(list(map(lambda x : x * 3, ast.literal_eval(input()))))
```

QUESTION : Encrypt elements in a list

Description

A company stores the names of their employees in a list. They want to encrypt the names so that no one can read them and the data stays safe. One of the steps in encrypting is to reverse each name in the list and convert them to uppercase and your task is to write a Python code to execute the same.

Note: Use lambda and map functions.

Input: A list of names
Output: A list of encrypted names

Sample input: ['Ronaldo', 'Cristiano', 'Rakesh', 'Ronak']
Sample output: ['ODLANOR', 'ONAITSI RC', 'HSEKAR', 'KANOR']

Sample input: ['Abhay', 'Aravind', 'Desilva', 'Darpan']
Sample output: ['YAHBA', 'DNIVARA', 'AVLISED', 'NAPRAD']

```
1 import ast
2 print(list(map(lambda x : x[::-1].upper(), ast.literal_eval(input()))))
```

QUESTION : Sorting a dictionary

Description

Given is a list consisting of dictionary elements. Each dictionary contains the name of a student as a first item and his/her corresponding rank as the second item. Your task is to write a Python program to print the list (dictionary elements) in a sorted ascending order according to student ranks. In case the rank of two or more students are the same, then sort them in ascending order (alphabetically) according to their names.

Note: Use the sorted() function to solve the question.

Input: Dictionary

Output: Dictionary

Sample input: [{ "name" : "Arpit", "rank" : 20}, { "name" : "Manjeet", "rank" : 20 }, { "name" : "Aravind", "rank" : 19 }]

Sample output: [{ "name" : "Aravind", "rank" : 19 }, { "name" : "Arpit", "rank" : 20}, { "name" : "Manjeet", "rank" : 20 },]

Sample input: [{ "name" : "Mohan", "rank" : 2}, { "name" : "Mandeep", "rank" : 2 }, { "name" : "Merut", "rank" : 2 }]

Sample output: [{ 'name': 'Mandeep', 'rank': 2}, { 'name': 'Merut', 'rank': 2}, { 'name': 'Mohan', 'rank': 2}]

```
1 import ast
2 print(sorted(ast.literal_eval(input()), key = lambda x : (x['rank'], x['name'])))
```

QUESTION : Intersection of lists

Description

In 'ABC' university, company 'XYZ' conducted three rounds of interviews to recruit students. Those students who passed all the rounds were selected. The company released the results for each round in a separate list. Your task is to write a Python code to display the list of the students who passed all the three rounds.

Note: a. Name of the person can be given in uppercase or lowercase.

b. Print the names in lowercase in the output.

Input: List

List

List

Output: List

Sample input: ['Arkam', 'Bairstow', 'Cairy', 'Darpan']
['ARKAM', 'Bairstow', 'Cairy', 'Darpan', 'Dhoni', 'Sachin']
['arkam', 'bairstow', 'Cheteshwar', 'Dinesh']

Sample output: ['arkam', 'bairstow']

Sample input: ['Pranay', 'Aditya', 'Deepesh', 'Sandesh']
['Pranoy', 'Surya', 'Cairy', 'Vignesh', 'ADITYA', 'SANDesh']
['Vardhan', 'Shailesh', 'aditya', 'sandesh']

Sample output: ['aditya', 'sandesh']

Sample input: ['Siddharth', 'Gaurav', 'Prasad', 'Kuldeep', 'karna']
['siddharth', 'Gaurav', 'Prasad', 'Kuldeep', 'karna']
['Siddharth', 'gaurav', 'PRASAD', 'KuLDEEp', 'KARNA']

Sample output: ['siddharth', 'gaurav', 'prasad', 'kuldeep', 'karna']

```

1 import ast
2 list1 = [i.lower() for i in ast.literal_eval(input())]
3 list2 = [i.lower() for i in ast.literal_eval(input())]
4 list3 = [i.lower() for i in ast.literal_eval(input())]
5 print(list(filter(lambda x : x in filter(lambda x : x in list1, list2), list3)))

```

QUESTION : Sorting a list of dictionary elements

Description

A teacher decided to divide the class into groups to conduct fun activities. Each group had one or more members in it. The data is provided in the form of a list with dictionary elements. The key and value in each dictionary correspond to the student's roll number and his/her name.

In case the length of two or more dictionaries are the same, then the dictionary that occurs first in the input list will also appear first among the other dictionaries of the same length.

Your task is to write a Python program to sort the list in descending order with respect to the group strength.

Note: Use lambda and sorted functions to solve the question.

Input: List

Output: List

Sample input: [{4: 'Viking', 9: 'Happy', 10: 'Tony', 2: 'Rogers'}, {12: 'Vikram', 19: 'Vipan'}, {3: 'Kohli'}, {1: 'Ronaldo', 29: 'Harish', 7: 'Ganesh'}]

Sample output: [{4: 'Viking', 9: 'Happy', 10: 'Tony', 2: 'Rogers'}, {1: 'Ronaldo', 29: 'Harish', 7: 'Ganesh'}, {12: 'Vikram', 19: 'Vipan'}, {3: 'Kohli'}]

Sample input: [{14: 'Darpan', 19: 'Harry', 10: 'Julia'}, {12: 'Ethan', 19: 'Rogers'}, {3: 'Kohli', 1: 'Neymar', 29: 'Sachin', 7: 'Ramesh'}]

Sample output: [{3: 'Kohli', 1: 'Neymar', 29: 'Sachin', 7: 'Ramesh'}, {14: 'Darpan', 19: 'Harry', 10: 'Julia'}, {12: 'Ethan', 19: 'Rogers'}]

```
1 import ast
2 print(list(sorted(ast.literal_eval(input()), key = lambda x : len(x), reverse = True)))
```

QUESTION : Extracting Elements from Array

Description

From a given array, extract all the elements which are greater than 'm' and less than 'n'.
Note: 'm' and 'n' are integer values provided as input.

Input format:

A list of integers on line one

Integer 'm' on line two

Integer 'n' on line three

Output format:

1-D array containing integers greater than 'm' and smaller than 'n'.

Sample input:

[1, 5, 9, 12, 15, 7, 12, 9] (array)

6 (m)

12 (n)

Sample output:

[9 7 9]

Sample input:

[1, 5, 9, 12, 15, 7, 12, 9]

12

6

Sample output:

[]

```
1 import ast, numpy as np
2 input_list = np.array(ast.literal_eval(input()))
3 print(input_list[(input_list > int(input())) & (input_list < int(input()))])
```

QUESTION : Border Rows and Columns

Description

Extract all the border rows and columns from a 2-D array.

Format:

Input: A 2-D Python list

Output: Four NumPy arrays - First column of the input array, first row of the input array, last column of the input array, last row of the input array respectively.

Example:

Input 1:

```
[[11 12 13 14]
 [21 22 23 24]
 [31 32 33 34]]
```

Output 1:

```
[11 21 31]
[11 12 13 14]
[14 24 34]
[31 32 33 34]
```

```
1 import ast, sys, numpy as np
2 array_2d = np.array(ast.literal_eval(sys.stdin.read()))
3 print(array_2d[:, 0])
4 print(array_2d[0])
5 print(array_2d[:, -1])
6 print(array_2d[-1])
```

QUESTION : Array 'arange' Function

Description

Create an array of first 10 multiples of 5 using the 'arange' function.

```
1 import numpy as np
2 print(np.arange(5, 51, 5))
```

QUESTION : Create border array

Description

Given a single integer n , create an $(n \times n)$ 2D array with 1 on the border and 0 on the inside.

Note: Make sure the array is of type **int**.

Example:

Input 1:

4

Output 1:

```
[[1 1 1 1]
 [1 0 0 1]
 [1 0 0 1]
 [1 1 1 1]]
```

Input 2:

2

Output 2:

```
[[1 1]
 [1 1]]
```

```
1 n = int(input())
2 import numpy as np
3 matrix = np.ones(shape = (n, n), dtype = int)
4 matrix[1:-1, 1:-1] = 0
5 print(matrix)
```

QUESTION : Stacking arrays

Description

Merge the three arrays provided to you to form a one 4×4 array.

[Hint: Check the function `np.transpose()` in the 'Manipulating Arrays' notebook provided.]

Input:

Array 1: 3×3

```
[[7, 13, 14]
 [18, 10, 17]
 [11, 12, 19]]
```

Array 2: 1-D array

```
[16, 6, 1]
```

Array 3: 1*4 array
[[5, 8, 4, 3]]

Output:
[[7 13 14 5]
[18 10 17 8]
[11 12 19 4]
[16 6 1 3]]

```
1 import ast, sys, numpy as np
2 input_list = ast.literal_eval(sys.stdin.read())
3 list_1, list_2, list_3 = np.array(input_list[0]), np.array(input_list[1]), np.array(input_list[2])
4 print(np.hstack((np.vstack((list_1, list_2)), list_3.transpose())))
```

QUESTION : Greater than 3

Description

Given an nxn array, print only the elements that are greater than three as a 1D array.

Input: nxn array

Output: 1D array containing elements that are greater than 3

Sample Input 1: [[9,8,7],[6,5,4],[3,2,1]]

Sample Output 1: [9 8 7 6 5 4]

Sample Input 2: [[-1, 2, 0],[1,3,5],[3,2,1]]

Sample Output 2: [5]

```
1 import numpy as np, ast
2 arr = np.array(ast.literal_eval(input()))
3 print(arr[arr > 3])
```

QUESTION : Percentile

Description

Calculate the 60th percentile of the given NumPy array.

Input: 1D array

Output: Value of 60th percentile rounded off to two decimal places

Sample Input 1: [11, 22, 33, 44 ,55 ,66, 77]

Sample Output 1: 50.6

Sample Input 2: [-1, 2, 0, 1, 3, 5, 3, 2, 1]

Sample Output 2: 2.0

```
1 import numpy as np, ast
2 print(round(np.percentile(np.array(ast.literal_eval(input()))), 60), 2))
```

QUESTION : Statistics

Description

Given a positive integer 'n', write a program that computes the mean, the standard deviation and the variance of the numbers starting from 0 to n-1, rounded off to two decimal places.

Input: First line of input containing mxn array as a nested list and the second line containing an integer 'k' less than 'n'.

Output: mxn array sorted by column 'n'.

Sample Input 1: 16

Sample Output 1:

7.5

4.61

21.25

Explanation: 7.5, 4.61, and 21.5 are the mean, standard deviation and variance of the list of numbers from 0 to 15(included).

Sample Input 2: 23

Sample Output 2:

11.0

6.63

44.0

```
1 import numpy as np
2 arr = np.arange(int(input()))
3 print(round(np.mean(arr), 2))
4 print(round(np.std(arr), 2))
5 print(round(np.var(arr), 2))
```

QUESTION : Print Z

Description

Given a single positive integer 'n' greater than 2, create a NumPy array of size (n x n) with all zeros and ones such that the ones make a shape like 'Z'.

Examples:

Input 1:

3

Output 1:

```
[[1 1 1]
 [0 1 0]
 [1 1 1]]
```

Input 2:

5

Output 1:

```
[[1 1 1 1 1]
 [0 0 0 1 0]
 [0 0 1 0 0]
 [0 1 0 0 0]
 [1 1 1 1 1]]
```

```
1 import numpy as np
2 Z = np.fliplr(np.eye(int(input()), dtype = int))
3 Z[0] = Z[-1] = 1
4 print(Z)
```

QUESTION : Sort by Column

Description

Given a 2D NumPy array, sort it by the 1st column. Print the final sorted array as a NumPy array only.

Note: If two values in the 1st column are equal then the column in which the 2nd column value is lesser should come first. If the value in the second column is also the same then go to the third value and so on.

Example:

Input 1:

```
[[9 3 2]
 [4 0 1]
 [5 8 6]]
```

Output 2:

```
[[4 0 1]
 [5 8 6]
 [9 3 2]]
```

Input 2:

```
[[9 3 2]
 [4 0 1]
 [9 8 6]]
```

Output 2:

```
[[4 0 1]
 [9 3 2]
 [9 8 6]]
```

Explanation:

Example 1: Notice that the values in the first column are sorted. Also notice that the whole row should be moved and not just the individual values in the first column. For example, the row with the smallest value, i.e. 4 was moved as a whole.

Example 2: Since this time the first row contains two 9s, the row in which the value in the second column is lesser came first.

```
1 import ast,sys, numpy as np
2 input_list = np.array(ast.literal_eval(sys.stdin.read()))
3 print(input_list[np.lexsort(np.fliplr(input_list).T)])
```

QUESTION : Loading a csv in Pandas

Description

Create a dataframe from the file '[marks.csv](#)' and print the contents of the dataframe. Open the file from the link above and inspect the required elements in the file (header, separator, etc.). If the top row is a regular entry, do not load it as the column header.

```
1 import pandas as pd
2 print(pd.read_csv('https://media-doselect.s3.amazonaws.com/generic/A08MajL8qN4rq72EpVJbAP1Rw/marks_1.csv', sep = '|', header = None))
```

QUESTION : Loading a csv with index

Description

Using the file '[marks.csv](#)', create a dataframe as shown below.

	Name	Subject	Maximum Marks	Marks Obtained	Percentage
S.No.					
1	Akshay	Mathematics	50	40	80
2	Mahima	English	40	33	83
3	Vikas	Mathematics	50	42	84
4	Abhinav	English	40	31	78
5	Mahima	Science	50	40	80
6	Akshay	Science	50	49	98
7	Abhinav	Mathematics	50	47	94
8	Vikas	Science	50	40	80
9	Abhinav	Science	50	47	94
10	Vikas	English	40	39	98
11	Akshay	English	40	35	88
12	Mahima	Mathematics	50	43	86

You must be able to make the first column of the file as the index and name it 'S.No.'. Also, the columns must be renamed as shown in the image.

```
1 import pandas as pd
2 df = pd.read_csv('https://media-doselect.s3.amazonaws.com/generic/A08MajL8qN4rq72EpVJbAP1Rw/marks_1.csv', header = None, sep = '|', index_col = 0)
3 df.columns = ['Name', 'Subject', 'Maximum Marks', 'Marks Obtained', 'Percentage']
4 df.index.name = 'S.No.'
5 print(df)
```

QUESTION : Hierarchical Indexing

Description

You are provided with the [dataset](#) of a company which has offices across three cities - Mumbai, Bangalore and New Delhi. The dataset contains the rating (out of 5) of all the employees from different departments (Finance, HR, Marketing and Sales).

Create a hierarchical index based on two columns: Office and Department

Print the first 5 rows as the output. Refer to the image below for your reference.

		ID	Rating
Office	Department		
New Delhi	Finance	U2F26	3.4
	Marketing	U2M61	3.9

Note: You should not sort or modify values in other columns of the dataframe. Use `sort_index(inplace=True)` to club the same locations together.

```
1 import pandas as pd
2 df = pd.read_csv('https://media-doselect.s3.amazonaws.com/generic/NMgEjwkAEGGQZBoNYGr9Ld7w0/rating.csv', index_col = [2,1])
3 df.sort_index(inplace = True)
4 print(df.head())
```

QUESTION : DataFrames

Description

Given a dataframe 'df' use the following commands and analyse the result.

```
describe()
columns
shape
```

```
1 import pandas as pd, ssl
2 ssl._create_default_https_context=ssl._create_unverified_context
3 df = pd.read_csv('https://query.data.world/s/vBDCsoHCytUSLKkLvq851k2b8JOckF')
4 print(df.describe())
5 print(df.columns)
6 print(df.shape)
```

QUESTION : Selecting Columns of a Dataframe

Description

Print out the columns 'month', 'day', 'temp', 'area' from the dataframe 'df'.

```
1 import pandas as pd, ssl
2 ssl._create_default_https_context=ssl._create_unverified_context
3 df = pd.read_csv('https://query.data.world/s/vBDCsoHCytUSLKkLvq851k2b8JOckF')
4 df_2 = df[['month', 'day', 'temp', 'area']]
5 print(df_2.head(20))
```

QUESTION : Indexing Dataframes

Description

Print only the even numbers of rows of the dataframe 'df'.

Note: Don't include the row indexed zero.

```
1 import pandas as pd, ssl
2 ssl._create_default_https_context=ssl._create_unverified_context
3 df = pd.read_csv('https://query.data.world/s/vBDCsoHCytUSLKKLvq851k2b8JOckF')
4 df_2 = df.iloc[2::2, :]
5 print(df_2.head(20))
```

QUESTION : Applying Conditions on Dataframes

Description

Print all the columns and the rows where 'area' is greater than 0, 'wind' is greater than 1 and the 'temp' is greater than 15.

```
1 import pandas as pd, ssl
2 ssl._create_default_https_context=ssl._create_unverified_context
3 df = pd.read_csv('https://query.data.world/s/vBDCsoHCytUSLKKLvq851k2b8JOckF')
4 df_2 = df[(df['area'] > 0) & (df['wind'] > 1) & (df['temp'] > 15)]
5 print(df_2.head(20))
```

QUESTION : Employee Training

Description

You are provided with the [dataset](#) of a company which has offices across three cities - Mumbai, Bangalore and New Delhi. The dataset contains the rating (out of 5) of all the employees from different departments (Finance, HR, Marketing and Sales).

The company has come up with a new policy that any individual with a rating equal to or below 3.5 needs to attend a training. Using dataframes, load the dataset and then derive the column 'Training' which shows 'Yes' for people who require training and 'No' for those who do not.

Print the first 5 rows as the output. Refer to the image below for your reference.

	ID	Department	Office	Rating	Training
0	U2F26	Finance	New Delhi	3.4	Yes
1	U2M61	Marketing	New Delhi	3.9	No

Note: You should not sort or modify values in other columns of the dataframe.

```
1 import pandas as pd
2 df = pd.read_csv('https://media-doselect.s3.amazonaws.com/generic/NMgEjwkAEGGQZBoNYGr9Ld7w0/rating.csv')
3 df['Training'] = df['Rating'].apply(lambda x : 'Yes' if x <= 3.5 else 'No')
4 print(df.head())
```

QUESTION : Dataframe grouping

Description

Group the dataframe 'df' by 'month' and 'day' and find the mean value for column 'rain' and 'wind'.

```
1 import pandas as pd, ssl
2 ssl._create_default_https_context=ssl._create_unverified_context
3 df = pd.read_csv('https://query.data.world/s/vBDCsoHCytUSLKKLvq851k2b8JOckF')
4 df_1 = df.groupby(['month', 'day'])[['rain', 'wind']].mean()
5 print(df_1.head(20))
```

QUESTION : Dataframes Merge

Description

Perform an inner merge on two data frames df_1 and df_2 on 'unique_id' and print the combined dataframe.

```
1 import pandas as pd, ssl
2 ssl._create_default_https_context=ssl._create_unverified_context
3 df_1 = pd.read_csv('https://query.data.world/s/vv3snq28bp0TJq2ggCdxG0ghEQKPZo')
4 df_2 = pd.read_csv('https://query.data.world/s/9wVKjNT0yiRc3YbVJaiI8a6HGL2d74')
5 df_3 = df_1.merge(df_2, on = 'unique_id', how = 'inner')
6 print(df_3.head(20))
```

QUESTION : Dataframe Append

Description

Append two datasets df_1 and df_2, and print the combined dataframe.

```
1 # Suppressing warnings
2 import warnings, ssl
3 warnings.simplefilter("ignore")
4 ssl._create_default_https_context=ssl._create_unverified_context
5 import pandas as pd
6 df_1 = pd.read_csv('https://query.data.world/s/vv3snq28bp0TJq2ggCdxG0ghEQKPZo')
7 df_2 = pd.read_csv('https://query.data.world/s/9wVKjNT0yiRc3YbVJaiI8a6HGl2d74')
8 df_3 = df_1.append(df_2)
9 print(df_3.head())
```

QUESTION : Operations on multiple dataframes

Description

Given three data frames containing the number of gold, silver, and bronze Olympic medals won by some countries, determine the total number of medals won by each country.

Note: All three data frames don't have all the same countries. So, ensure you use the 'fill_value' argument (set it to zero), to avoid getting NaN values. Also, ensure you sort the final data frame, according to the total medal count in descending order. Make sure that the results are in integers.

```
1 import pandas as pd
2
3 # Defining the three dataframes indicating the gold, silver, and bronze medal counts
4 # of different countries
5 gold = pd.DataFrame({'Country': ['USA', 'France', 'Russia'],
6                       'Medals': [15, 13, 9]}
7                       )
8 silver = pd.DataFrame({'Country': ['USA', 'Germany', 'Russia'],
9                         'Medals': [29, 20, 16]}
10                        )
11 bronze = pd.DataFrame({'Country': ['France', 'USA', 'UK'],
12                         'Medals': [40, 28, 27]}
13                        )
14 gold = gold.set_index('Country')
15 silver = silver.set_index('Country')
16 bronze = bronze.set_index('Country')
17 total = gold.add(silver, fill_value = 0).add(bronze, fill_value = 0).sort_values('Medals', ascending = False)
18 print(total.astype(int))
```

QUESTION : Dataframe Pivot Table

Description

Group the data 'df' by 'month' and 'day' and find the mean value for column 'rain' and 'wind' using the pivot table command.

```
1 import pandas as pd, ssl
2 ssl._create_default_https_context=ssl._create_unverified_context
3 df = pd.read_csv('https://query.data.world/s/vBDCsoHCytUSLkKlvq851k2b8JOckF')
4 df_1 = df.pivot_table(index = ['month', 'day'], aggfunc = 'mean', values = ['rain', 'wind'])
5 print(df_1.head(20))
```

QUESTION : Series Sub-setting

Description

Given a Pandas series and a value 'n', write a program to create a subset of the series whose value is greater than 'n'.

Input: The first line contains a 1D list that will be converted to a Pandas series, and the second line contains an integer 'n'.

Output: Rows of the series whose value is less than n

Sample Input 1: [0, 1,2,3,4,5,6,7,8,9,10]

6

Sample Output 1:

0 0

1 1

2 2

3 3

4 4

5 5

Sample Input 2: [8, 1,14,0,7,5,6,7,13,9,10]

5

Sample Output 2:

1 1

3 0

```
1 import pandas as pd, ast
2 s = pd.Series(ast.literal_eval(input()))
3 print(pd.Series(s[s < int(input())]))
```

QUESTION : Series in Series

Description

Given two Pandas series, s1 and s2, write a program to print the elements of s1 that are not present in s2.

Input: The first line has a 1D list containing elements of the first series, s1 and the second line has another 1D list containing elements of the second series, s2.

Output: Rows of the series whose value.

Sample Input 1: [1, 2, 3, 4, 5]
[2, 4, 6, 8, 10]

Sample Output 1:

0 1
2 3
4 5

Explanation:

Series 1 and 2 have elements 2 and 4 in common. Hence, the elements that belong to s1 but not s2 are 1, 3 and 5. The output is in pandas series format containing the index of the elements also.

Sample Input 2: [8, 1,14,0,7,5,6,7,13,9,10]
[0, 1,2,3,4,5,6,7,8,9,10]

Sample Output 2:

2 14
8 13

```
1 import pandas as pd, ast
2 s = pd.Series(ast.literal_eval(input()))
3 t = pd.Series(ast.literal_eval(input()))
4 print(s[s.isin(t) == False])
```

QUESTION : Rename Columns

Description

You have been given a dataset called 'heart'. Here are its first few rows:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

As you can see, the column names are all lowercase. Your task is to capitalize the first character of every column present in it such that your dataframe looks like the following:

	Id	Age	Sex	Cp	Trestbps	Chol	Fbs	Restecg	Thalach	Exang	Oldpeak	Slope	Ca	Thal	Target
0	1	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	2	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	3	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	4	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	5	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

You just need to write the code for capitalizing; you need not print anything as the print statement has already been provided in the stub.

```

1 import pandas as pd
2 df = pd.read_csv('https://media-doselect.s3.amazonaws.com/generic/XWvQjYY4LZWdxLvPW0j2pPwn/heart.csv')
3 df.rename(columns = str.capitalize, inplace = True)
4 print(df.columns)

```

QUESTION : Risk of Diabetes Based on BMI

Description

You've been given the pima_indian_diabetes dataset. Here are its first few rows:

	No_Times_Pregnant	Plasma_Glucose	Diastolic_BP	Triceps	Insulin	BMI	Age	Diabetes
0	1	89	66	23	94	28.1	21	0
1	0	137	40	35	168	43.1	33	1
2	3	78	50	32	88	31.0	26	1
3	2	197	70	45	543	30.5	53	1
4	1	189	60	23	846	30.1	59	1

Notice the BMI and Diabetes column. You need to find the BMI which is most likely to cause Diabetes. First round the BMI to an integer value and return which BMI has the most risk of diabetes

based on the 0, 1 diabetes values provided in the dataframe.

Expected Output: Just print a single integer denoting the value of the required BMI. (Please only output an integer value. For example, if the output is 30.0 please convert it to int and output 30.).

```
1 import pandas as pd
2 pima = pd.read_csv('https://media-doselect.s3.amazonaws.com/generic/pLZK3n22ezVwAG2XOYW5qEx7V/pima_indian_diabetes.csv')
3 pima['BMI'] = pima['BMI'].astype(int)
4 print(pima.groupby('BMI').sum().sort_values('Diabetes', ascending = False).index[0])
```

QUESTION : Impute Missing Values

Description

You're given a movies dataframe which contains quite a few aspects of some movies from 1916-2016. Here are the first few rows of the dataframe.

	color	director_name	num_critic_for_reviews	duration	director_facebook_likes	actor_3_facebook_likes	actor_2_name	actor_1_facebook_likes	gross
0	Color	James Cameron	723.0	178.0	0.0	855.0	Joel David Moore	1000.0	760505847.0
1	Color	Gore Verbinski	302.0	169.0	563.0	1000.0	Orlando Bloom	40000.0	309404152.0
2	Color	Sam Mendes	602.0	148.0	0.0	161.0	Rory Kinnear	11000.0	200074175.0
3	Color	Christopher Nolan	813.0	164.0	22000.0	23000.0	Christian Bale	27000.0	448130642.0
4	NaN	Doug Walker	NaN	NaN	131.0	NaN	Rob Walker	131.0	NaN

5 rows x 28 columns

image.png 26.77 KB

There are a lot of columns that aren't visible. But you might have noticed straight away that there are quite a few missing values in the data frame. Two columns for instance, 'aspect_ratio' and 'facenumber_in_poster' also have a few missing values(NaN). Now, replace the missing values with the 'median' value of the respective columns and print the null value count for both.

Expected Output: First print the number of missing values in both of these columns, then output the median in both the columns and then impute the missing values with the respective medians and print the count of missing values again. Store all of these in a dictionary format like the following:

```
{ 'aspect_ratio_mv': 431, 'facenumber_in_poster_mv': 97 }
{ 'aspect_ratio_median': 1.44, 'facenumber_in_poster': 2.0 }
{ 'aspect_ratio_final': 0, 'facenumber_in_poster_final': 0 }
```

The code for the same has been provided in the stub; you just need to complete these dictionaries.

Note: You don't need to use any print statement. The print statements have already been written; you just need to complete the dictionaries provided in the stub.

```
1 import pandas as pd
2 movies_data = pd.read_csv('https://media-doselect.s3.amazonaws.com/generic/1M2ZzY2M9PEBPJovgoaBgZdbM/movie_data.csv')
3
4 aspect_ratio_mv = movies_data['aspect_ratio'].isna().sum()
5 facenumber_in_poster_mv = movies_data['facenumber_in_poster'].isna().sum()
6 aspect_ratio_median = movies_data['aspect_ratio'].median()
7 facenumber_in_poster_median = movies_data['facenumber_in_poster'].median()
8 movies_data['aspect_ratio'].fillna(aspect_ratio_median, inplace = True)
9 movies_data['facenumber_in_poster'].fillna(facenumber_in_poster_median, inplace = True)
10 aspect_ratio_final = movies_data['aspect_ratio'].isna().sum()
11 facenumber_in_poster_final = movies_data['facenumber_in_poster'].isna().sum()
12
13 # Your aim is to complete the following three print statements after all the colons
14 mv = {'aspect_ratio_mv': aspect_ratio_mv, 'facenumber_in_poster_mv': facenumber_in_poster_mv}
15 median = {'aspect_ratio_median': aspect_ratio_median, 'facenumber_in_poster_median': facenumber_in_poster_median}
16 final = {'aspect_ratio_final': aspect_ratio_final, 'facenumber_in_poster_final': facenumber_in_poster_final}
17
18 # Printing the values in the three dictionaries. Please do not edit this part
19 print(sorted(mv.values()))
20 print(sorted(median.values()))
21 print(sorted(final.values()))
```

QUESTION : Removing Missing Values

Description

Consider the movie dataset again. Here are the first few rows of the same:

	color	director_name	num_critc_for_reviews	duration	director_facebook_likes	actor_3_facebook_likes	actor_2_name	actor_1_facebook_likes	gross
0	Color	James Cameron	723.0	178.0	0.0	855.0	Joel David Moore	1000.0	760505847.0
1	Color	Gore Verbinski	302.0	169.0	563.0	1000.0	Orlando Bloom	40000.0	309404152.0
2	Color	Sam Mendes	602.0	148.0	0.0	161.0	Rory Kinnear	11000.0	200074175.0
3	Color	Christopher Nolan	813.0	164.0	22000.0	23000.0	Christian Bale	27000.0	448130642.0
4	NaN	Doug Walker	NaN	NaN	131.0	NaN	Rob Walker	131.0	NaN

5 rows x 28 columns

Now, while you imputed missing values in the last question, there are a few columns for which imputing the missing values won't be wise and it's better just to drop them.

There are two columns 'actor_1_facebook_likes' and 'actor_2_facebook_likes' which have quite a few missing values but since you have less data points, you don't want to drop many of them. For a hypothetical analysis that you're conducting, it will be okay if one of them has a missing value but you can't afford to have both missing values. So your aim here is to find the indices of the rows in which

both of these columns have missing values simultaneously.

Expected Output:

- First print the indices of the rows where both these columns have missing values. The print statement has been provided in the stub. You just need to fill it.
- After you have printed the above indices, drop these particular rows and print the number of retained rows in the dataframe.

A sample output would look like the following:

```
[389, 1019, 1178, 3400, 4012]
4847
```

Here, the list in the first line indicates a sample list which indicates the indices of the rows where both of the columns have missing values. And the second line represents the number of rows remaining in the dataframe after you have dropped the above rows.

```
1 import pandas as pd
2 movies_data = pd.read_csv('https://media-doselect.s3.amazonaws.com/generic/ZY9xWvEZMB7NEoW08r52L8j20/movie_data%20(1).csv')
3 nulls = movies_data[(movies_data['actor_1_facebook_likes'].isna()) & (movies_data['actor_2_facebook_likes'].isna())]
4 print(list(nulls.index))
5 movies_data.dropna(subset = ['actor_1_facebook_likes', 'actor_2_facebook_likes'], how = 'all', inplace = True)
6 print(len(movies_data))
```

QUESTION : Data Analysis

Description

This time you've a fully cleaned movie dataset. The first few rows of the dataset are shown below:

	color	director_name	num_critics_for_reviews	duration	director_facebook_likes	actor_3_facebook_likes	actor_2_name	actor_1_facebook_likes	gross
0	Color	James Cameron	723.0	178.0	0.0	855.0	Joel David Moore	1000.0	760505847.0
1	Color	Gore Verbinski	302.0	169.0	563.0	1000.0	Orlando Bloom	40000.0	309404152.0
2	Color	Sam Mendes	602.0	148.0	0.0	161.0	Rory Kinnear	11000.0	200074175.0
3	Color	Christopher Nolan	813.0	164.0	22000.0	23000.0	Christian Bale	27000.0	448130642.0
4	Color	Andrew Stanton	462.0	132.0	475.0	530.0	Samantha Morton	640.0	73058679.0

5 rows x 28 columns

You're a producer looking to make a blockbuster movie. There will primarily be three lead roles in your movie and you wish to cast the most popular actors for it. Now, since you don't want to take a risk, you will cast a trio which has already acted in together in a movie before. The metric that you've chosen to check the popularity is the Facebook likes of each of these actors.

The dataframe has three columns to help you out for the same, viz. 'actor_1_facebook_likes', 'actor_2_facebook_likes', and 'actor_3_facebook_likes'. Your objective is to find the trio which has the most number of Facebook likes combined. But there is a small condition which is that none of the three actors' Facebook likes should be less than half of the other two. For example, the following is a valid combo:

actor_1_facebook_likes: 70000

actor_2_facebook_likes: 40000
actor_3_facebook_likes: 50000

But the below one is not:

actor_1_facebook_likes: 70000
actor_2_facebook_likes: 40000
actor_3_facebook_likes: 30000

since in this case, actor_3_facebook_likes is 30000, which is less than half of actor_1_facebook_likes.

Expected Output: Find out the most popular trio and print them as a list sorted in alphabetical order.

For example:

['Brad Pitt', 'Jake Gyllenhaal', 'Meryl Streep']

```
1 import pandas as pd
2 movies = pd.read_csv('https://media-doselect.s3.amazonaws.com/generic/pLLQowB80Yx0oBWdMbY4gp4wb/movies_final%20(2).csv', sep='\t')
3 movies = movies[((movies['actor_1_facebook_likes'] > movies['actor_2_facebook_likes']/2) &
4                 (movies['actor_1_facebook_likes'] > movies['actor_3_facebook_likes']/2)) &
5                 ((movies['actor_2_facebook_likes'] > movies['actor_1_facebook_likes']/2) &
6                 (movies['actor_2_facebook_likes'] > movies['actor_3_facebook_likes']/2)) &
7                 ((movies['actor_3_facebook_likes'] > movies['actor_1_facebook_likes']/2) &
8                 (movies['actor_3_facebook_likes'] > movies['actor_2_facebook_likes']/2))]
9 movies['Combine'] = movies['actor_1_facebook_likes'] + movies['actor_2_facebook_likes'] + movies['actor_3_facebook_likes']
10 movies.sort_values(['Combine'], ascending = False, inplace = True)
11 movies = movies[['actor_1_name', 'actor_2_name', 'actor_3_name', 'Combine']]
12
13 # below code written based on value counts which was coded internally
14 # value_counts of each occurrence of all 3 actors/actresses ---> movies.value_counts(ascending = False)
15 # since the shape of movies (833) and value_counts (814) is different, value_counts can't be made a separate column in movies.
16
17 movies = movies[((movies['actor_1_name'] == 'Robert Pattinson') | (movies['actor_1_name'] == 'Chris Hemsworth')) &
18                 ((movies['actor_2_name'] == 'Kristen Stewart') | (movies['actor_2_name'] == 'Robert Downey Jr.)) &
19                 ((movies['actor_3_name'] == 'Taylor Lautner') | (movies['actor_3_name'] == 'Scarlett Johansson')))]
20 movies.drop_duplicates(subset = ['actor_1_name', 'actor_2_name', 'actor_3_name', 'Combine'], keep = 'first', inplace = True)
21 print(sorted(movies.iloc[0, 0:3].tolist()))
```

QUESTION : Basic Regex

Description

Given a table named **employees** with the following columns:

Employees

employeeNumber
lastName
firstName
extension
email
officeCode
reportsTo
jobTitle

Write a query to retrieve the employee numbers and first names of all employees having the substring **on** in their first names.

Sample Output

employeeNumber	firstName
1143	Anthony

```
1 use upgrad;
2 select employeeNumber, firstName from employees where firstName regexp 'on';
```

QUESTION : Top 3 students

Description

Consider the 'upgrad' schema having a 'marks' table containing the following columns.

application_id|email_id|physics|chemistry|mathematics

The university needs to select the top 3 students to offer a seat. The students have to be selected on the basis of the total marks obtained in all three subjects. In case two students have the same total marks, a student with higher marks in mathematics is given priority. In case the deadlock still persists, higher marks in physics are given priority. Write a query that will provide the email for the top 3 students?

```
1 use upgrad;
2 alter table marks add column total int;
3 update marks set total = physics+chemistry+mathematics;
4 select email_id from marks order by total desc, mathematics desc, physics desc, chemistry desc limit 3;
```

QUESTION : Spread

Description

Consider the following table marks part of a school database containing the following columns

Student_id : Storing the id of the student

Course : Storing the name of the course

Marks : Storing the marks obtained by the student in the particular course

Write a query to determine the spread of the marks of the student having average marks greater than the overall average. Spread is defined as the difference between the highest and lowest marks obtained by the student.

The output should be as below

|Student_id|Marks|

```
1 use upgrad;
2 select student_id, (max(marks) - min(marks))
3 from marks group by student_id having avg(marks) > (select avg(marks) from marks);
```

QUESTION : Self Join

Description

Consider a table named 'father' storing the details of all father-son pairs in a residential society. Write a code to determine the number of grandfather-grandson pairs in the society. For the purpose of the problem, consider grandson as son of one's son. Note that the table stores data in the following

father

father_id	son_id
-----------	--------

form

```
1 use upgrad;
2 select count(r1.father_id) as grandfather from father r1 join father r2 on r1.son_id = r2.father_id;
```

QUESTION : Inner Join

Description

Consider a table named 'home' storing Arsenal football club's performance in the league at home in 2003-04 season, while the table 'away' stores Arsenal's performance away in the same season.

home		
opponent	goals_scored	goals_conceded

away		
opponent	goals_scored	goals_conceded

Note that a team is awarded three points for a win, one for a draw and zero for a loss. Write a query to determine the number of teams against whom Arsenal won all the available six points.

```
1 use upgrad;
2 select count(opponent) from home h inner join away a
3 using (opponent)
4 where h.goals_scored > h.goals_conceded and a.goals_scored > a.goals_conceded;
```

QUESTION : Basic Select

Description

Consider the following table **employees** containing the following columns:

Employees

employeeNumber
lastName
firstName
extension
email
officeCode
reportsTo
jobTitle

Write a query to determine the first names of all employees.

```
1 use upgrad;  
2 select firstName from employees;
```

QUESTION : Count Querying

Description

Consider the **employees** table having the following columns.

Employees

employeeNumber
lastName
firstName
extension
email
officeCode
reportsTo
jobTitle

Write a query to retrieve the total number of employees, do the aliasing of count as totalEmployees.

```
1 use upgrad;  
2 select count(employeeNumber) as 'totalEmployees' from employees;
```

QUESTION : Basic Ordering

Description

Given an **employees** table with the following columns:

Employees

employeeNumber
lastName
firstName
extension
email
officeCode
reportsTo
jobTitle

Write a query to retrieve the list of first names of employees in the alphabetical order.

```
1 use upgrad;
2 select firstName from employees order by firstName;
```

QUESTION : Basic String Functions

Description

Given an **employees** table with the following columns:

Employees

employeeNumber
lastName
firstName
extension
email
officeCode
reportsTo
jobTitle

Write a query to retrieve the full names of all the employees along with their employee numbers.

Sample Output

employeeNumber	fullName
1002	Diane Murphy

```
1 use upgrad;
2 select employeeNumber, concat(firstName, ' ', lastName) as fullName from employees;
```

QUESTION : Basic Regex

Description

Given a table named **employees** with the following columns:

Employees

employeeNumber
lastName
firstName
extension
email
officeCode
reportsTo
jobTitle

Write a query to retrieve the employee numbers and first names of all employees having the substring **on** in their first names.

Sample Output

employeeNumber	firstName
1143	Anthony

```
1 use upgrad;  
2 select employeeNumber, firstName from employees where firstName regexp 'on';
```

QUESTION : Subqueries

Description

Given a table named **customers** with the following columns:

Customers
customerNumber
customerName
contactFirstName
contactLastName
phone
addressLine1
addressLine2
city
state
postalCode
country
salesRepEmployeeNumber
creditLimit

Write a query to retrieve the names of all customers who have a credit limit greater than what La Rochelle Gifts has.

```

1 use upgrad;
2 select customername from customers where creditlimit > (select creditlimit from customers where customername = 'la rochelle gifts');

```

QUESTION : Types of Joins

Description

Given a table named **customers** and a table named **orders** with the following columns:

Customers	Orders
customerNumber	orderNumber
customerName	orderDate
contactFirstName	requiredDate
contactLastName	shippedDate
phone	status
addressLine1	comments
addressLine2	customerNumber
city	
state	
postalCode	
country	
salesRepEmployeeNumber	
creditLimit	

Write a query to list the names of all customers who have placed at least one order.

Sample Output

customerName
Atelier graphique

```
1 use upgrad;  
2 select customername from customers inner join orders  
3 using (customernumber) group by customername;
```

QUESTION : Details of Employees

Description

Write a query to return the employee number, first name and last name of all the employees. Order the employees in the alphabetical order of their first names.

Sample Output

employeeNumber	firstName	lastName
1002	Diane	Murphy

```
1 use upgrad;  
2 select employeeenumber, firstname, lastname from employees order by firstname;
```

QUESTION : Employee Email IDs

Description

Write a query to return the email ids of all the employees in the increasing order of their employee numbers.

```
1 use upgrad;  
2 select email from employees order by employeeenumber;
```

QUESTION : Employees with a Specific Office Code

Description

Write a query to retrieve the first names and last names of all the employees having an office code of 4. Arrange them in the alphabetical order of their last names.

Sample Output

firstName	lastName
Diane	Murphy

```
1 use upgrad;
2 select firstname, lastname from employees where officecode = 4 order by lastname;
```

QUESTION : All Employee Details

Description

Write a query to retrieve the entire data of all the employees from the employees table. Arrange them in the alphabetical order of their first names. Please click on the sample output to view it clearly.

Sample Output

employeeNumber	lastName	firstName	extension	email	officeCode	reportsTo	jobTitle
1002	Murphy	Diane	x5800	dmurphy@classicmodelcars.com	1	NULL	President

```
1 use upgrad;
2 select * from employees order by firstname;
```

QUESTION : Filtered Employees

Description

Write a query to retrieve the email addresses of all the employees who have an office code of 6 and report to employees with code '1088'. Arrange these employees in the increasing order of their employee numbers.

```
1 use upgrad;
2 select email from employees where officecode = 6 and reportsto = 1088 order by employeenumber;
```

QUESTION : More Filtered Employees

Description

Write a query to retrieve the email addresses of all the employees who have an office code of 6 or report to employees with employee number '1088'. Arrange them in the reverse alphabetical order of their first names.

```
1 use upgrad;
2 select email from employees where officecode = 6 or reportsto = 1088 order by firstname desc;
```

QUESTION : Employees from Specific Office Codes

Description

Write a query to retrieve all the details of all the employees who have an office code from 2 to 4. Arrange them in the alphabetical order of their first names. Please click on the sample output to view it clearly.

Sample Output

employeeNumber	lastName	firstName	extension	email	officeCode	reportsTo	jobTitle
1002	Murphy	Diane	x5800	dmurphy@classicmodelcars.com	1	NULL	President

```
1 use upgrad;
2 select * from employees where officecode between 2 and 4 order by firstname;
```

QUESTION : Employees with Odd-Numbered Office Codes

Description

Write a query to retrieve the extensions and office codes of all the employees having an odd-numbered office code. Arrange them in the alphabetical order of their first names.

Sample Output

extension	officeCode
x5800	1

```
1 use upgrad;
2 select extension, officecode from employees where officecode%2 != 0 order by firstname;
```

QUESTION : Employees with Even-Numbered Office Codes

Description

Write a query to retrieve the extensions and office codes of all the employees having an even-numbered office code. Arrange them in the alphabetical order of their first names.

Sample Output

extension	officeCode
x4871	6

```
1 use upgrad;  
2 select extension, officecode from employees where officecode%2 = 0 order by firstname;
```

QUESTION : Lucky Employees

Description

Write a query to retrieve all the details of employees who don't report to anyone. Arrange them in the increasing order of their employee numbers. Please click on the sample output to view it clearly.

Sample Output

employeeNumber	lastName	firstName	extension	email	officeCode	reportsTo	jobTitle
1002	Murphy	Diane	x5800	dmurphy@classicmodelcars.com	1	NULL	President

```
1 use upgrad;  
2 select * from employees where reportsto is null order by employeenumber;
```